

STAGE 0-5 / EXPERIMENTAL PREDESIGN

Code Agent KVCache

Experimental Predesign Report

Effective-Reuse Frontiers and Tiered Retention for Pause-Resume Coding Agents

Exact-Prefix Reuse and Value-Density-Driven Tiered KV Retention

Review decision Section 0-5.7 was approved by the project owner on September 2, 2026. The research questions, goals, hypotheses, workloads, baselines, experiment matrix, primary metrics, correctness gates, and reproducibility rules are frozen as of this version.

Document field	Frozen value
Stage	Stage 0-5: experimental predesign and Stage 1 admission
Version	v1.0 / Frozen
Date	September 2, 2026
Initial environment	Ubuntu / RTX 5090 32 GB / 64 GB DRAM / local NVMe
Initial model	Qwen2.5-Coder-7B family; exact revision pending the Stage 1 environment audit
Initial engine	vLLM; exact version pending the compatibility audit
Document type	Research and experimental design; contains no completed performance results

Frozen research-design edition | Editable master document

Document Control and Reading Notes

Item	Description
Status	0-5.7 approved; Stage 0 content frozen; research-repository closure follows archival of this report.
Scope	Governs the Stage 1 environment audit, Pilot and Core experiments, correctness validation, and result archival.
Change control	Frozen items are not to be changed in response to observed results. Any necessary change must record its rationale, version, impact, and renewed approval.
Data discipline	Pilot data do not enter Core results; failures and negative findings are retained; candidate targets must not be presented as measurements.
External sharing	This report may explain the research plan to supervisors or collaborators. Unpublished prompts, traces, secrets, and unsanitized logs must not enter external copies.

Executive Summary

This project studies how to reuse KVCache safely during the multi-turn, long-context, tool-interleaved reasoning of Code Agents. Unlike ordinary chat sessions, coding agents pause while executing commands, reading files, waiting for tests, or awaiting human confirmation. They then usually resume by appending relatively few tokens, while return times, session concurrency, and context lengths all exhibit long tails. This structure creates substantial reuse potential, but cache value can be erased by GPU-memory pressure, migration overhead, or an incorrect identity match.

Stage 0 narrows the broad idea of prefix caching plus generic tiered storage into a falsifiable main line. In a constrained single-node environment with an RTX 5090 32 GB GPU, 64 GB of DRAM, and local NVMe, the first-phase correctness boundary is exact token-prefix reuse. The study will measure the gap between theoretical reuse and effective reuse, then make value-density-driven joint decisions among GPU, DRAM, NVMe, and recomputation.

Core evidence chain Why reuse -> what can be reused safely -> where to restore it from -> whether restoration is faster than recomputation -> whether the output is correct -> whether the task succeeds -> whether the conclusion is reproducible.

Frozen Decisions at a Glance

Dimension	Frozen decision	Rationale
Research object	Pause-resume, append-only Code Agent sessions	Focuses on realistic reuse structure that can be validated on one node
Main line	Exact prefix plus joint GPU/DRAM/NVMe/recompute decisions	Correctness first; benefits must include the complete restoration path
Core questions	RQ1-RQ4	Reusable volume, effective boundary, tier selection, and task-level value
Long-term question	RQ5	Generalization across models, attention designs, hardware, and workloads
High-risk extension	G6 non-prefix reuse	Begins only after the exact-prefix main line passes all gates
Primary metrics	Correctness, effective hits, P95 TTFT, session completion time, and cost per successful task	Avoids substituting a single hit-rate number for system value
Experimental discipline	Separate Pilot/Core, paired traces, and retain negative results	Reduces selection bias and supports reproducibility

1. Stage Positioning and Research Boundaries

1.1 Purpose of Stage 0-5

The purpose of 0-5 is not to implement a complete caching system immediately. It converts the converged research problems from 0-4 into an executable, comparable, falsifiable, and reproducible experimental protocol. The protocol must define workloads, replay modes, baselines, experiment groups, metrics, correctness gates, statistical rules, exception handling, hardware boundaries, and Stage 1 admission conditions.

1.2 Research Object and System Events

A Code Agent task alternates between multiple LLM calls and tool executions. After each model call, the session may pause, at which point the system chooses whether to retain or migrate its KV state. When the session resumes, the system validates cache identity and integrity and restores from the selected tier. If the expected benefit is insufficient, the cache is absent, or the identity is incompatible, the system performs prefill again.

- Event boundaries: request arrival, prefill, decode, tool start, tool end, human wait, session resume, and session completion.
- Candidate actions: retain on GPU, migrate to DRAM, migrate to NVMe, or evict and recompute on return.
- Decision inputs: prefix length, KV bytes, predicted return time, tier latency/bandwidth, concurrency pressure, compatibility domain, and prediction confidence.
- Decision outputs: target tier, migration priority, TTL or re-evaluation time, fallback path, and auditable rationale.

1.3 Explicit First-Phase Exclusions

- Multi-node RDMA, distributed caches, and large-scale cluster scheduling.
- Treating access to H100/H200 or CXL/HBM servers as a condition for starting Stage 1.
- Direct KV reuse for arbitrarily semantically similar text.
- Sharing private content across tenants or accepting a cache hit without a complete compatibility key.
- Freezing CUDA, PyTorch, vLLM, or model revisions before an environment audit.
- Treating a provider's prompt-cache internals as a local KV-tensor experiment.

2. Research Questions, Goals, and Falsifiable Hypotheses

2.1 Research Questions (RQs)

ID	Research question	Primary observation
RQ1	How much KVCache in Code Agent requests can be reused safely?	Exact-prefix structure, compatible hits, and resident hits
RQ2	How large is the gap between theoretically reusable data and effective reuse that delivers a positive benefit?	Logical hits, negative-benefit hits, and effective hits
RQ3	How should the system make joint decisions among GPU, DRAM, NVMe, and recomputation?	Break-even boundaries, capacity pressure, migration, and queueing overhead
RQ4	Does tiered reuse improve P95 TTFT, session completion time, and cost per successful task?	Performance, success rate, energy, and cost
RQ5	Do the core findings generalize across models, architectures, hardware, and Agent workloads?	Relative rankings, boundary shifts, and failure conditions

2.2 Research Goals (G1-G8)

Goal	Description	Priority
G1	Characterize stable exact prefixes and safely reusable volume	Main line
G2	Quantify the gaps among theoretical, resident, and effective hits	Main line
G3	Build a joint GPU/DRAM/NVMe/recompute decision policy	Main line
G4	Connect system benefits to task value through cost per successful task	Main line
G5	Evaluate layouts that place stable prompt content first	Main line
G6	Explore dependency-aware non-prefix reuse with selective recomputation	High-risk extension
G7	Establish multi-tenant, version, integrity, and security boundaries	Main-line gate
G8	Evaluate generalization across models, attention designs, hardware, and tasks	Long-term extension

2.3 Falsifiable Hypotheses (H1-H8)

ID	Testable statement	Falsification condition
H1	Code Agent traces contain substantial stable exact token prefixes.	Prefix repetition in W1/W2 is low and B1 can reuse almost nothing.
H2	The theoretical hit rate is materially higher than the effective hit rate.	The gap remains negligible across pressure conditions.
H3	Value-density-driven B5 outperforms strong baselines under equal correctness and cache budgets.	B5 cannot consistently improve the frozen primary metrics.
H4	Measurable benefit boundaries exist among DRAM, NVMe, and recomputation.	One tier is always superior or always ineffective in every condition.
H5	Placing stable content first improves safe exact-prefix reuse.	Hits do not improve, or task quality/token identity declines.
H6	Dependency awareness and selective recomputation enable controlled non-prefix reuse.	Errors are unacceptable, or repair costs erase all benefits.
H7	Complete compatibility keys, trust-domain salts, and integrity checks prevent incorrect sharing.	Cross-tenant, cross-version, or corrupted KV is accepted.
H8	The core relationships apply to more than one model or hardware configuration.	The conclusions depend entirely on one device, model, or workload.

Freeze discipline Hypotheses are allowed to be falsified. Primary metrics, hypotheses, and exception rules must not be redefined after Core results are observed merely to obtain a positive finding.

3. Method Definitions and Benefit Model

3.1 Three Types of Hit

Concept	Decision rule	Counts as effective
Theoretical/ logical hit	An identical exact token prefix exists in history.	No
Resident hit	A compatible cache remains readable in a storage tier.	No
Effective hit	The complete restoration path succeeds, is faster than recomputation, and passes correctness gates.	Yes
Negative-benefit hit	A logical hit is restored successfully, but the restoration path is not faster than recomputation.	No; reported separately

3.2 Timing Decision

Effective-reuse condition $T_{\text{lookup}} + T_{\text{queue}} + T_{\text{transfer}} + T_{\text{restore}} < T_{\text{recompute}}$

Every component must be measured on the same hardware, at the same concurrency, and across identical timing boundaries. NVMe tests must distinguish cold reads, page-cache-hot reads, and explicitly controlled conditions so that Linux page-cache DRAM performance is not misclassified as storage performance.

3.3 Candidate Value Function

Candidate decision expression $\text{Benefit}(i,l) = p_{\text{return}}(i) \times [T_{\text{recompute}}(i) - T_{\text{restore}}(i,l)] - C_{\text{move}}(i,l) - \lambda_l \times \text{Bytes}(i)$

Here, i denotes a cache entry or block and l denotes a storage tier. This expression is a candidate framework to be tested through measurements and ablations; it is not a proven algorithm. If no tier yields a positive benefit, the system selects recomputation. Under capacity constraints, entries with greater expected benefit per byte receive priority.

3.4 Frozen Metric Definitions

- Theoretical token hit rate $H_{\text{token}} = \text{sum of logically reused prefill tokens} / \text{sum of all prefill tokens}$.
- Effective token hit rate $H_{\text{effective}} = \text{sum of correct tokens whose complete restoration path is faster than recomputation} / \text{sum of all prefill tokens}$.
- Cost per successful task $C_{\text{success}} = \text{total end-to-end system cost} / \text{number of successful tasks}$.

4. Workload and Replay Design

4.1 Three Workload Classes

Workload	Content	Primary value	Primary limitation
W1 Public real traces	Request lengths, timestamps, session relations, tool pauses, and concurrency	Realistic structure and distributions for R1	May omit full prompts, preventing exact output reproduction
W2 Self-collected SWE-bench Agent traces	A fixed Agent/model/template/tool/task/decoding setup with complete token IDs and timestamps	Supports R2/R3, task success, and cost per successful task	Expensive to collect and requires strict version control
W3 Controlled synthetic traces	Independently varies length, pause, concurrency, cache pressure, edit ratio, or return probability	Isolates causal variables and benefit boundaries	Does not represent complete real behavior

4.2 Three Replay Modes

Mode	Preserved content	Purpose	Correctness requirement
R1 Structural replay	Timing, lengths, sessions, dependencies, and request order	Scheduling, capacity, and tier pressure	Event and resource behavior must match
R2 Token-exact replay	Identical token IDs and request boundaries	KV correctness and repeatable performance	Exact-prefix output tokens must be identical
R3 End-to-end replay	The Agent, tools, tasks, and tests run for real	Task value, success rate, and total cost	Task tests and success definitions must be frozen

4.3 Minimum Trace Schema

Field group	Required fields
Identity	trace_id, session_id, request_id, tenant_domain, experiment_run_id
Model compatibility	model_revision, tokenizer_hash, adapter_id, engine_version, kv_layout, dtype, quantization
Timing	arrival, prefill_start/end, decode_end, tool_start/end, resume, finish
Tokens	input_token_ids/hash, prefix_length, output_token_ids/hash, position/RoPE configuration
Cache events	cache_key_hash, tier_before/after, bytes, lookup/queue/transfer/restore time, fallback_reason
Task outcome	task_id, tests_passed, success, retry_count, failure_class
System state	GPU/CPU/DRAM/NVMe, temperature, frequency, power, swap, I/O, and concurrency

5. Baselines and Fair-Comparison Protocol

Baseline	Definition	Question answered
B0	No cross-request cache; prefill is recomputed every time	What is the theoretical maximum saving?
B1	Exact prefix plus LRU	Does the proposed method beat a mature conventional prefix-cache baseline?
B2	Fixed TTL	Does a complex policy beat a simple retention window?
B3	Return-probability prediction	Does value density improve on prediction alone?
B4	DRAM only	Do NVMe and joint decisions add value?
B5	Joint GPU + DRAM + NVMe + recompute policy	Does the proposed method improve the frozen primary metrics?
B6	Provider keepalive/prompt cache	Independent economic and user-experience comparison; excluded from local performance ranking

5.1 Frozen Fairness Conditions

- Use the same model and weights, tokenizer, prompt tokens, output length, and decoding configuration.
- Use the same request order, hardware, concurrency, total cache budget, warm-up, and measurement window.
- Use paired traces for B0-B5; randomize or interleave run order to limit time drift.

- Align cache budgets by usable bytes; no policy receives extra GPU, DRAM, or NVMe capacity.
- Report B6 pricing, TTFT, task success, and limitations separately; do not rank its opaque infrastructure directly against local implementations.

5.2 B5 Decision Outputs and Auditability

Every B5 action must record the estimated benefit for each candidate tier, the selected action, predicted return probability, bytes, migration cost, final restoration path, fallback status, and rationale. The policy must produce auditable decisions and may not hide incorrect hits or budget overruns behind unexplained global state.

6. Experiment Matrix M0-M8

Experiment	Core task	Entry condition	Exit evidence
M0 Smoke test	Connect the model, trace, metrics, and manifest	Environment audit complete	Minimal request is reproducible and the metric pipeline is complete
M1 Microbenchmark	Measure GPU/DRAM/NVMe restoration and recomputation curves	M0 passed	Break-even curves by token count, bytes, and concurrency
M2 Synthetic causal study	Vary length, pause, concurrency, and pressure independently	M1 boundaries available	Main and interaction effects are interpretable
M3 Real structural replay	Replay W1 under R1	Trace-validity checks passed	Capacity and scheduling results under real distributions
M4 Token-exact replay	Replay W2 under R2	B0/B1 correctness passed	Output tokens match and paired comparison is possible
M5 End-to-end Agent	Run W2 under R3	Task harness frozen	Success rate, completion time, and cost per successful task
M6 Ablation	Disable B5 components one at a time	B5 runs reliably	Actual contributions and adverse effects identified
M7 Security and faults	Inject wrong keys, corruption, cross-tenant access, and I/O/process faults	Compatibility key and checks implemented	Incorrect states are rejected and safely recomputed
M8 Generalization	Evaluate other models, architectures, hardware, and tasks	Core main line complete	General relationships and device-specific conclusions identified

6.1 M1 Must Precede Complex Policies

M1 directly measures GPU hits, DRAM restoration, cold and hot NVMe restoration, and fresh prefill across token lengths, KV byte sizes, and concurrency levels. Its output is a break-even curve, not an assumption that NVMe must be useful. If NVMe is slower than recomputation throughout every feasible region on the current device, the formal result should record that ineffective boundary, and B5 may degrade to a GPU/DRAM/recompute policy.

6.2 M6 Ablation Set

- No NVMe: determines whether the third tier contributes positive value.
- No return-probability predictor: measures the marginal value of prediction.
- No value density: replaces it with a conventional capacity/time policy.
- No asynchronous migration: measures the contribution of transfer overlap.
- No prompt normalization/layout optimization: measures G5's contribution.
- LRU replaces the joint policy: compares B5 directly with the strongest simple policy.

6.3 Recommended Execution Order

Step	Action	Gate
1	Conduct a read-only environment inventory	Do not modify drivers or existing environments
2	Create an isolated private experiment repository and environment	Keep it separate from existing projects such as cuSZp
3	Verify minimal model inference and the trace schema	Produces M0
4	Implement B0/B1 and strict correctness validation	Exact token identity
5	Execute M1	Freeze the method for measuring benefit boundaries
6	Implement B2-B4, followed by B5	Retain strong baselines at every step
7	Run W3/W1/W2 Pilots	Select budgets, concurrency, repetitions, and timeouts
8	Freeze Core parameters	Pilot data do not enter Core
9	Execute M2-M7 Core experiments	Paired, randomized, and fully logged
10	Proceed to G6/M8 as resources allow	Must not block the main-line conclusion

7. Metrics, Correctness, and Statistical Protocol

7.1 Primary and Supporting Metrics

Level	Metrics	Purpose
Hard gates	Cache identity, shape/dtype/checksum, logit tolerance, output tokens, and task correctness	Determines whether a performance result is eligible for analysis
Primary metrics	Effective token hit rate, P95 TTFT, session completion time, and cost per successful task	Supports RQ2-RQ4
Supporting metrics	Theoretical/resident hits, P50/P99, prefill savings, transfer bytes/time, fallback rate, GPU utilization, energy, and retries	Explains mechanisms and tail behavior

7.2 Correctness Gates

- Input tokens must be identical.
- Model, tokenizer, adapter, position/RoPE configuration, dtype, quantization, KV layout, engine, and kernel versions must be compatible.
- Cache shape, length, checksum, and atomic-write state must be valid.
- Exact-prefix experiments: any output-token mismatch is a hard failure.
- Logit or hidden-state comparisons are diagnostic and do not replace the token-level hard gate.
- End-to-end experiments must report task tests, success, and retries; text similarity alone is insufficient.
- Any run that fails a correctness gate is excluded from performance conclusions.

7.3 Pilot and Core

Phase	May determine	Prohibited
Pilot	Safe cache capacity, concurrency range, warm-up length, repetitions, timeout, noise, and exception thresholds	Must not be merged with Core or presented as the final effect
Core	Formal comparisons under frozen parameters	Must not change primary metrics, baselines, timeouts, or exception rules while inspecting results

7.4 Statistical Plan

- Use paired comparisons over the same traces for B0-B5; randomize or interleave run order.
- Report sample count, repetitions, median, and P95, together with confidence intervals and effect sizes.
- Use robust methods appropriate to the distributions of TTFT, completion time, and cost; retain request-level raw results.
- Report full hits, partial hits, misses, negative-benefit hits, and fallbacks separately by tier.
- Before Core, use Pilot data to freeze the minimum detectable effect and repetition count; do not select slices after the fact.
- The earlier 70%, 95%, 90%, and 10% figures remain candidate targets, not achieved results.

8. Security, Isolation, and Exception Handling

8.1 Compatibility Keys and Trust Domains

Category	Must bind
Model identity	Model name, exact revision, weight hash, and tokenizer hash
Execution identity	Engine version, kernel version, attention implementation, and KV layout
Numerical configuration	dtype, quantization, layer count, KV heads, and head dimension
Position semantics	Position IDs, RoPE/scaling configuration, and prefix-token length
Extensions	LoRA, adapter, prompt template, and system-instruction version
Security domain	Tenant, project, trust domain, independent salt per domain, and access permissions

8.2 Integrity and Privacy

- Use cryptographic hashes for cache keys and an independent salt for each trust domain.
- Protect cache files with least-privilege access, checksums, and atomic writes.
- Sanitize logs and traces; do not record plaintext that can reconstruct private prompts or stable cross-domain identifiers.
- M7 must cover cross-tenant and cross-version access, wrong dtype/layout, truncated files, bit flips, partial writes, and stale indexes.

8.3 Exception Classification

Exception	Handling	How counted
Normal cache miss	Recompute the current request	Normal system behavior
Identity/checksum failure	Reject the cache and recompute safely	Fallback and security event
Method-caused OOM/timeout	Abort or fall back while retaining evidence	Method failure; must not be hidden as invalid
Driver restart, swap, severe throttling, or external-process interference	Invalidate the entire run batch	Environment invalid; use a new run_id after repair
Corrupted KV or incorrect hit	Stop the affected experiment immediately and diagnose	Correctness hard failure

9. Hardware, Software, and Capacity Plan

9.1 First-Phase Hardware Boundary

Resource	Known condition	Stage 1 measurements
GPU	RTX 5090, 32 GB GDDR7	Driver, usable memory, frequency, power, PCIe, and temperature
System memory	64 GB	Actually usable memory, NUMA, bandwidth, swap, and page-cache effects
NVMe	Local device; capacity and model pending audit	File system, free space, sequential/random I/O, small/large blocks, concurrency, and cold/hot P95/P99
CPU	Pending audit	Model, core count, memory-copy, integrity-check, and I/O overhead

An initial DRAM-cache candidate range of roughly 24-40 GB may be considered, but it must not be fixed in advance. Capacity must remain available for Ubuntu, Python, the inference engine, the Agent, tokenizer, analysis, file-system page cache, and I/O buffers. At least roughly 200-300 GB of free NVMe space is recommended; more than 500 GB is more comfortable for long-term retention of several models, traces, and results, but the actual requirement will be set by the environment audit.

9.2 Software Stack and Timing

Technology	Purpose	Required at Stage 1 start
Python	Inference, traces, policies, experiments, and analysis	Required
Bash	Read-only checks and launch scripts	Limited use
YAML/JSON	Configuration, schema, and manifest	Required
Parquet/PyArrow	Request-level and event-level results	Recommended
DuckDB/SQL	Queries over large result sets	Recommended
PyTorch + vLLM	Initial inference and cache baselines	Required
pytest	Correctness, fault, and regression tests	Required
C++/CUDA	Deep KV management, kernels, or G6	Later, as needed
LaTeX/Markdown	Papers and reports	Later

10. Reproducibility and Repository Rules

10.1 Separate Experiment Repository

Stage 1 should use a separate private repository, `code-agent-kvcache-experiments`, isolated from this research repository and from existing projects such as `cuSZp`. The research repository stores surveys, decisions, and design documents. The experiment repository stores code, configurations, schemas, lightweight logs, and reproducible result indexes. Large KV files, model weights, secrets, and unsanitized prompts must not enter Git.

10.2 Manifest for Every Run

Field group	Recorded content
Version	Code commit, dirty-worktree state, configuration hash, and schema version
Model	Model revision, weight/tokenizer hashes, adapter, dtype, and quantization
Environment	Ubuntu, driver, CUDA, Python, PyTorch, vLLM, and kernel versions
Hardware	GPU, CPU, memory, NVMe, file system, and PCIe
Input	Trace ID/hash, task set, request order, and random seed
Method	B0-B6, parameters, cache budgets, concurrency, warm-up, and Core/Pilot status

Field group	Recorded content
Result	Primary/supporting metrics, correctness, success/failure/invalid reason, and output path

10.3 Never-Overwrite Rule

- Generate a unique `experiment_run_id` for every run.
- Do not delete or overwrite failed, invalid, or negative runs; update only their status and reason.
- After any fix, create a new `run_id`; never modify an old result to resemble the original run.
- Store raw event logs, derived tables, and figures in separate layers; derived results must be reproducible.
- After Core configuration is frozen, archive it by commit and content hash.

11. Stage 1 Admission Checklist

11.1 Stage 0 Closure Items

Admission item	Status	Evidence/action
0-1 through 0-3 research materials and IEEE survey	Complete	Existing research-repository materials
0-4 evidence-gap matrix and problem convergence	Complete	Recovered Excel, DOCX, and PDF
0-5.1 through 0-5.6 experimental predesign	Complete	Body of this report
0-5.7 final review and hypothesis freeze	Approved	Project-owner confirmation on September 2, 2026
Complete 0-5 DOCX/PDF	Generated in this cycle	Deliver after page-by-page QA
Final research-repository organization	Pending report confirmation	Update the single root README and archive the reports
stage-0-complete tag	Pending repository audit	Create last

11.2 First Read-Only Checks in Stage 1

- Exact Ubuntu version; NVIDIA driver; CUDA; Python.
- RTX 5090 recognition, memory, and health; actually available portion of 64 GB system memory; swap.

- NVMe model, file system, mount, free space, and basic I/O.
- Docker, PyTorch, vLLM, and existing virtual environments.
- Locations and isolation requirements of existing projects such as cuSZp.

Admission restriction Before the read-only inventory is complete: do not install software, upgrade drivers, modify an existing Python environment, run formal performance experiments, or create/overwrite existing project directories.

11.3 Parameters That Do Not Block Stage 0

- Exact model revision and PyTorch/CUDA/vLLM version combination.
- DRAM and NVMe cache capacity, KV block size, and concurrency.
- Warm-up, repetitions, timeouts, and minimum detectable effect.
- Actual NVMe benefit boundary and whether B6 will use a paid API.
- Availability of HBM servers, other GPUs, or multi-node resources.

12. Risks, Stop Conditions, and Decision Gates

Risk/observation	Mandatory action	Effect on the research conclusion
Exact-prefix output mismatch	Stop performance analysis and repair identity/restoration logic	The main line has not passed correctness
B1 matches or beats B5 on primary workloads	Retain the negative result and reassess B5 complexity	H3 may be falsified
NVMe is slower than recomputation everywhere	Mark the no-benefit region and allow policy degradation	H4 is partly falsified or its boundary is clarified
Prompt layout reduces task quality	Stop the G5 modification	H5 is falsified
G6 cannot preserve both correctness and benefit	Terminate G6 without affecting the exact-prefix main line	H6 is falsified
Single-node results do not reproduce across devices	State device-specific conclusions explicitly	H8 is limited or falsified
Insufficient time or budget	Prioritize the minimum M0-M7 main-line evidence loop	Defer G6/G8

12.1 Stage 1 Entry Decision

Stage 0 closes only after this report passes visual QA, is archived in the research repository, and the stage-0-complete tag is created. Stage 1 then begins with the read-only environment audit. Model, framework, and cache parameters remain undecided until that audit and the Pilot; approval of this report does not authorize installation, upgrades, or experiments automatically.

13. Final 0-5.7 Freeze Decision

Formal decision Approve 0-5.7. Freeze RQ1-RQ5, G1-G8, H1-H8, W1-W3, R1-R3, B0-B6, M0-M8, the core metrics, correctness gates, statistical rules, security boundaries, and repository-isolation plan.

The Stage 0 technical design now forms a complete evidence loop, while project status remains pending report archival and repository closure. This report does not claim that any performance target has been achieved, and it does not prematurely freeze parameters that require a Stage 1 environment audit or Pilot measurements.

Appendix A: Index of Frozen Objects

Object	Scope	Location
RQ	RQ1-RQ5	Section 2.1
G	G1-G8	Section 2.2
H	H1-H8	Section 2.3
Workloads	W1-W3	Section 4.1
Replay	R1-R3	Section 4.2
Baselines	B0-B6	Section 5
Experiments	M0-M8	Section 6
Metrics/statistics	Primary metrics, gates, and Pilot/Core	Section 7
Security	Compatibility keys, integrity, and exceptions	Section 8
Admission	Stage 0 closure and Stage 1 checks	Section 11

Appendix B: Candidate Configuration Skeleton (Non-Frozen Values)

This skeleton exists only to keep Stage 1 configuration files complete. Values in angle brackets must be supplied by the environment audit or Pilot and must not be adopted directly as formal parameters.

Configuration group	Example fields
run	experiment_run_id, phase: <pilot core>, seed, repetitions
model	model_id, revision, tokenizer_hash, dtype, quantization
engine	vllm_version, block_size, max_model_len, attention_backend
workload	W1 W2 W3, R1 R2 R3, trace_hash, concurrency
cache	baseline B0-B5, gpu_budget, dram_budget, nvme_budget, ttl/predictor/value settings
measurement	warmup, timeout, clock source, power sample interval, cold/hot NVMe mode
correctness	token_exact, logit_tolerance, checksum, task_success_policy
output	manifest, event_log, request_table, summary, invalid_reason

Appendix C: Relationship to Stage 0-4 Outputs

The 0-4 report identified evidence gaps, scored candidate directions, and converged the research questions. Stage 0-5 expands those conclusions into workloads, replay modes, baselines, an experiment matrix, metrics, and an admission protocol. The 0-4 evidence matrix remains the primary index of external research evidence. This report therefore does not repeat the full 91-entry literature table and instead cites its converged conclusions. If directly competing work appears later, the evidence matrix should be updated before deciding whether a controlled revision of this report is necessary.

Appendix D: Principal References

- [1] K. Zhu et al., "TraceLab: Characterizing Coding Agent Workloads for LLM Serving," arXiv:2606.30560, 2026.
- [2] S. Tiwari et al., "CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents," arXiv:2606.16824, 2026.

- [3] H. Li et al., "Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live," arXiv:2511.02230, 2025/2026.
- [4] H. Kang et al., "ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System," arXiv:2602.13692, 2026.
- [5] S. Yu et al., "Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving," arXiv:2604.25899, 2026.
- [6] Y. Yao et al., "CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion," arXiv:2405.16444, 2024.
- [7] "EPIC: Efficient Position-Independent Caching for Serving Large Language Models," arXiv:2410.15332, 2024.
- [8] "LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference," arXiv:2510.09665, 2025.
- [9] "Strata: Hierarchical Context Caching for Long Context Language Model Serving," arXiv:2508.18572, 2025.
- [10] "KVDrive: A Holistic Multi-Tier KV Cache Management System for Long-Context LLM Inference," arXiv:2605.18071, 2026.
- [11] "Auditing Prompt Caching in Language Model APIs," arXiv:2502.07776, 2025.
- [12] NVIDIA, "GeForce RTX 5090 Specifications," 32 GB GDDR7.